Temasek POLYTECHNIC | School of Informatics & IT

COMT

Programming Principles IV

Week 05.

SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)

Overview

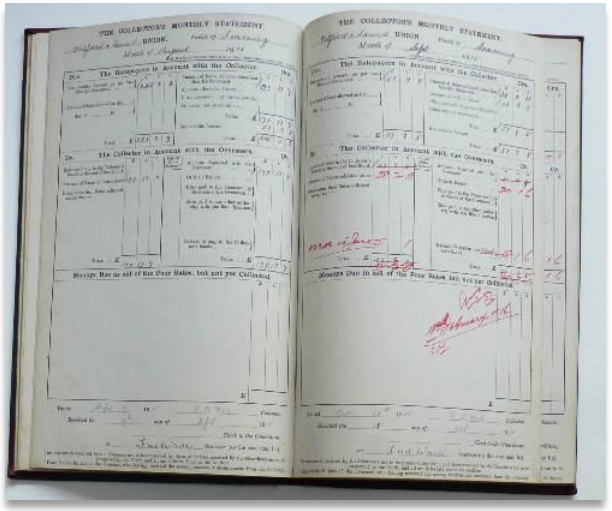
- Learn how to create object classes in Java.

Objectives

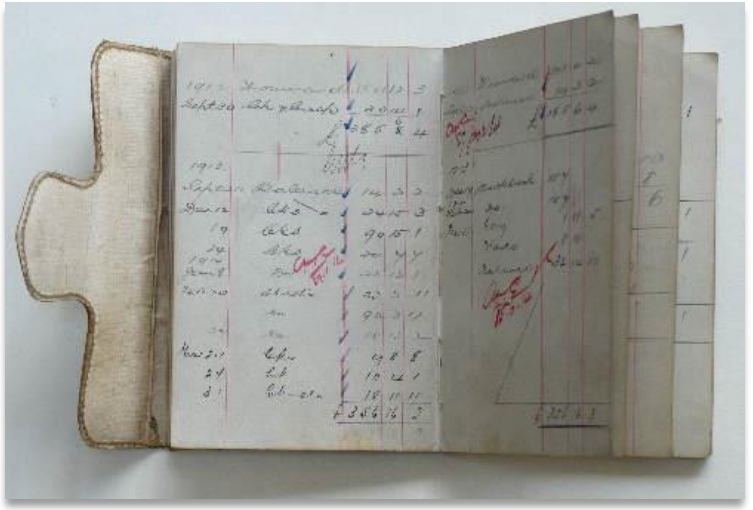
- Understand the concept of objects and classes in Java;
- Demonstrate how to create and use object classes in Java.

OBJECT ORIENTED PROGRAMMING

Before the Use of Computers…



Monthly statements of rates collected (1912 – 1920)



Old Barclays bank book. The page shown here shows all transactions in 1913!

- Before the use of computers, transferring money from one bank account to another required the use of paper and pen.
- Tons of papers were used to store information such as customers' bank account(s) details, transactions of money transferred, etc.
- It was a tedious, slow and painful job, and easily prone to human errors.

With the Use of Computers…

- With the use of computers, transferring money from one bank account to another requires a way to represent such real world process in digital form.
- There is no need for tons of papers to be used but there is still a need to store, manipulate and transfer data/information.
- There is no need for using pen to record information but there is now a need to write computer programs to automate the whole process of money transfer.
- **Object-Oriented Programming** provides a way to implement real world processes, such as the above, in the digital medium.

Object-Oriented Programming Languages

- There are many languages that supports Object-Oriented Programming.
- The following lists some of these languages:
  - Simula 67
  - Smalltalk
  - Java
  - C++
  - JavaScript
  - Python
  - C#

What is an Object?

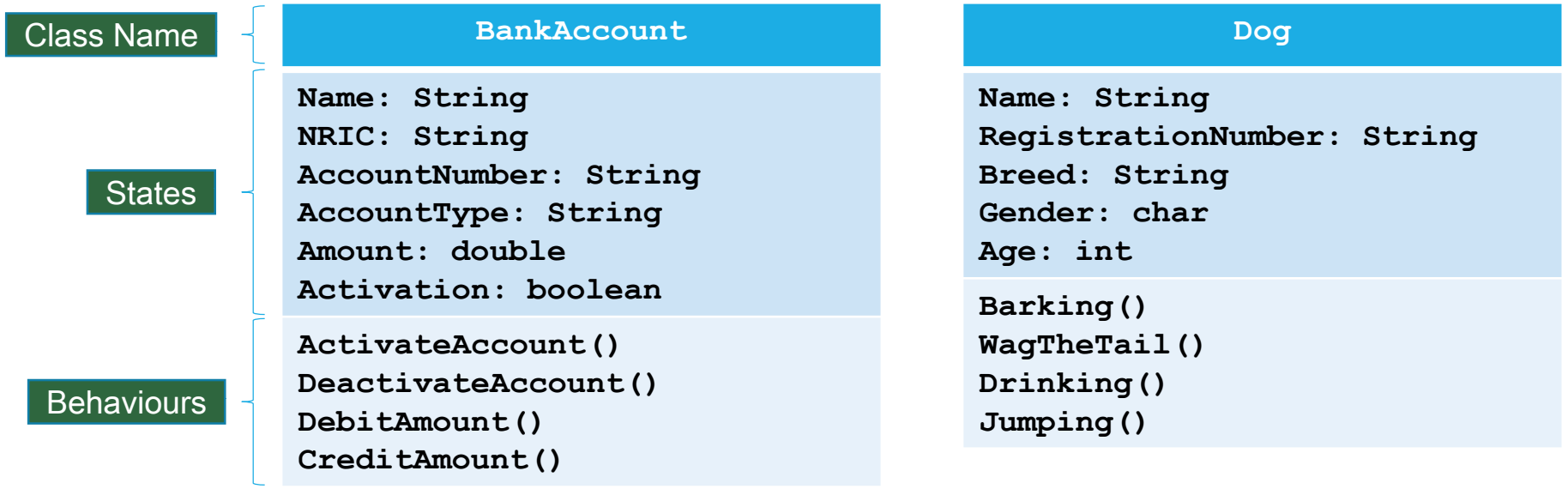
- An **Object** is something that we perceived in the real world that has **states** and **behaviours**.
- For instance, in the previous example a Bank Account is an object that could have the following states:
  - Customer Name
  - NRIC
  - Account Number
  - Account Type
  - Amount
  - Activation
- And the following behaviours:
  - Activate Account
  - Deactivate Account
  - Debit Amount
  - Credit Amount

What is an Object?

- Another example of an **Object** is a Dog.
- For instance, a Dog could have the following states:
  - Name
  - Breed
  - Gender
  - Age
- And the following behaviours:
  - Barking
  - Wag the tail
  - Drinking
  - Jumping

Class Diagram

- For documentation purpose, we can use a **Class Diagram** to represent Objects (so that it is easier to read) as shown in the following examples:



How to use Objects?

- In Object-Oriented Programming, we make use of Objects to group, transfer and manipulate data.
- As you can see from the previous examples, the data/information of Bank Account and Dog are contained in their respective states.
- The behaviours in each of them can be created to manipulate their data.
- So how do we create Objects in our program to use them?
- We will talk about this in the next section.

OBJECTS AND CLASSES IN JAVA

How to Create Objects?

- In Java, we use class declaration to implement objects.
- A **Class** is a blueprint for an Object that we write in code form.
- We represent the **states** and **behaviours** of an Object using a **Class** in code form.

How to Create Object States?

- Let's start with creating a **BankAccount** class and its states first.
- Create a new file called **BankAccount.java** and add the codes on the right into it.
- Object **states** are created using variables.
- Below the **states** is the constructor, which looks like a method.
- The `this` keyword is used here to refer to the class variables.

States

Constructor

```
public class BankAccount
{
    String name;
    String nric;
    String accountNumber;
    String accountType;
    double balance;
    boolean activation;

    public BankAccount(String _name, String _nric,
    String _accountNumber, String _accountType)
    {
        this.name = _name;
        this.nric = _nric;
        this.accountNumber = _accountNumber;

        this.accountType = _accountType;
        this.balance = 0.0;
        this.activation = false;
    }
}
```

What is a Constructor?

- The **Constructor** is written in the form of a method, except that it DOES NOT have a return type.
- It is first executed automatically when the class is being used. Hence, its main purpose is to initialize the states/variables of an object to certain value.
- The name of the **Constructor MUST** be the same as the Class name, even the case **MUST** be the same.
- There are two types of Constructor:
  - Default Constructor
  - Parameterized Constructor

What is a Constructor?

```
public BankAccount()
{
    System.out.println("This is a default constructor.");
}
```

Default constructor has no parameters, and provides the default values to the properties of the object such as `0.0` for `balance`, `null` for `name`, etc… depending on the type of the property.

```
public BankAccount(String _name, String _nric, String _accountNumber, String _accountType)
{
    this.name = _name;
    this.nric = _nric;
    this.accountNumber = _accountNumber;
    this.accountType = _accountType;
    this.balance = 0.0;
    this.activation = false;
}
```

Parameterized constructor is used to provide values to the properties of distinct objects.

How to Create Object Behaviours?

- Object **behaviours** are created using method / function.
- Now add the methods for the behaviours as shown on the right into the class.

Behaviours

```
1  public class BankAccount
2  {
3      String name;
4      String nric;
5      String accountNumber;
6      String accountType;
7      double balance;
8      boolean activation;
9
10     public BankAccount(String _name, String _nric, String _accountNumber,
       String _accountType)
11     {
12         this.name = _name;
13         this.nric = _nric;
14         this.accountNumber = _accountNumber;
15         this.accountType = _accountType;
16         this.balance = 0.0;
17         this.activation = false;
18     }
19
20     public void activateAccount()
21     {
22         this.activation = true;
23     }
24
25     public void debitAccount(double amountToDebit)
26     {
27         this.balance -= amountToDebit;
28     }
29
30     public void creditAccount(double amountToCredit)
31     {
32         this.balance += amountToCredit;
33     }
34
35     public double getBalance()
36     {
37         return this.balance;
38     }
39 }
```

Test It Out

- To test if your class works, it is good practice to create a separate testing file.
- Under the same folder where you saved the `BankAccount.java` file, create a new file called `TestBankAccount.java`.
- Add the codes on the right into `TestBankAccount.java`.
- To use a class object, we need to create an instance of it using the `new` keyword as shown in line 6.
- To use the methods in the `BankAccount` class, we use the dot operator "`.`" as shown in lines 8, 9 and 11.
- Compile and run the program `TestBankAccount` to see what happens.

```
1  public class TestBankAccount
2  {
3      public static void main(String args[])
4      {
5          //Create an instance of a BankAccount
6          BankAccount account1 = new BankAccount("John Loh", "S1234567D",
           "123-123-123-1", "Savings");
7
8          account1.activateAccount();
9          account1.creditAccount(5000.00);
10
11         double newBalance = account1.getBalance();
12         System.out.println("The balance is: $" + newBalance);
13     }
14 }
```

More Instance

- We can also create more instances of an object.
- For example, we can add another account as highlighted here. This is also called adding a new instance.
- Add the highlighted codes to `TestBankAccount.java`.
- Compile and run the program `TestBankAccount` to see what happens.

```
1  public class TestBankAccount
2  {
3      public static void main(String args[])
4      {
5          //Create an instance of a BankAccount
6          BankAccount account1 = new BankAccount("John Loh", "S1234567D",
           "123-123-123-1", "Savings");
7
8          BankAccount account2 = new BankAccount("Varun Dada", "S8888888A",
           "321-321-321-3", "Current");
9
10         account1.activateAccount();
11         account1.creditAccount(5000.00);
12         double newBalance = account1.getBalance();
13         System.out.println("The balance for account 1 is: $" + newBalance);
14
15         account2.activateAccount();
16         account2.creditAccount(25000.00);
17         double newBalance2 = account2.getBalance();
18         System.out.println("The balance for account 2 is: $" + newBalance2);
19     }
20 }
```

Review

- Notice that you do not need to use the static keyword in object class because it is not a program but an object.
- Before an object class can be used, an instance of it needs to be created with the `new` keyword. You can create many instances of an object.
- By just writing the codes of an object, you can re-use it many times.
- To use methods that are in any object classes, we use the dot operator "**.**".

Try It Out!

- Add additional functions to BankAccount.java to do the following:
  - Get the name of the account holder
  - Get the account type of the account holder
- In your TestBankAccount.java file, write the codes to debit $250.00 from the balance of **John Loh** in the example.
- Print out the name of the account holder, the account type and the balance to the console.

Challenge

- Add an additional function to **`BankAccount.java`** to transfer amount to another account. The function name can be "**`transferAmount`**".
- The function should have two parameters:
    1. Amount to transfer
    2. The account to transfer to
- In your `TestBankAccount.java` file, write the codes to create another instance of BankAccount with the following info:
    - Name: **`Elsa Goh`**
    - NRIC: **`S7654321D`**
    - Account Number: **`113-113-113-2`**
    - Account Type: **`Saving`**
- Activate Elsa Goh's account
- Credit $5000.00 into Elsa Goh's account
- Transfer $50.00 from John Loh's account to Elsa Goh's account.
- Print out the balance of both accounts to the console.

Summary

- In this lesson, you have learned the the basic concept of object and classes in object-oriented programming.
- In the next lesson, you will start to learn how to create Android application using Java.

Supplementary Materials

Topic	Source
Java Objects and Classes	https://www.tutorialspoint.com/java/java_object_classes.htm

Temasek POLYTECHNIC | School of Informatics & IT

COMT

THE END